

DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON

SYSTEM REQUIREMENTS SPECIFICATION
CSE 4316: SENIOR DESIGN I
SPRING 2026 (UPDATED SUMMER 2026)



AgriHome project logo: leaf and camera lens motif for vision-based agricultural monitoring.

AGRIHOME

GOVINDA AWASTHI
LAXMAN BHUSHAL
THUC DANG
SAGAR KHADKA
MUHAMMAD SAAD MEMON
ALEX TABLIZO
NATHAN NGUYEN TRAN

Revision History

Revision	Date	Author(s)	Description
0.1	03.2026	Team	Document creation from charter scope
0.2	03.2026	Team	Complete draft
0.3	03.2026	Team	Release candidate 1
1.0	03.2026	Team	Official release
1.1	04.2026	Team	Customer change requests
2.0	07.23.2026	Team	Summer update: edge device requirements 3.10–3.15; security & future iter
2.1	07.28.2026	Team	Klipper edge: fswebcam primary capture; klipper_url / updateKlipperUrl

Contents

1	Product Concept	10
1.1	Purpose and Use	10
1.2	Intended Audience	10
2	Product Description	10
2.1	Features & Functions	10
2.2	External Inputs & Outputs	10
2.3	Product Interfaces	11
3	Customer Requirements	11
3.1	Plant Health Status Display	11
3.1.1	Description	11
3.1.2	Source	11
3.1.3	Constraints	11
3.1.4	Standards	12
3.1.5	Priority	12
3.2	Automatic Image Capture	12
3.2.1	Description	12
3.2.2	Source	12
3.2.3	Constraints	12
3.2.4	Standards	12
3.2.5	Priority	12
3.3	Environmental Data Collection	12
3.3.1	Description	12
3.3.2	Source	12
3.3.3	Constraints	13
3.3.4	Standards	13
3.3.5	Priority	13
3.4	Alert Display	13
3.4.1	Description	13
3.4.2	Source	13
3.4.3	Constraints	13
3.4.4	Standards	13
3.4.5	Priority	13
3.5	Historical Data Display	13
3.5.1	Description	13
3.5.2	Source	13
3.5.3	Constraints	14
3.5.4	Standards	14
3.5.5	Priority	14
3.6	User Settings Input	14
3.6.1	Description	14
3.6.2	Source	14
3.6.3	Constraints	14
3.6.4	Standards	14
3.6.5	Priority	14

3.7	Dashboard Interface	14
3.7.1	Description	14
3.7.2	Source	14
3.7.3	Constraints	14
3.7.4	Standards	15
3.7.5	Priority	15
3.8	No Physical Plant Actuation	15
3.8.1	Description	15
3.8.2	Source	15
3.8.3	Constraints	15
3.8.4	Standards	15
3.8.5	Priority	15
3.9	Remote Notification Support	15
3.9.1	Description	15
3.9.2	Source	15
3.9.3	Constraints	15
3.9.4	Standards	15
3.9.5	Priority	16
3.10	Edge Device Auto-Provisioning	16
3.10.1	Description	16
3.10.2	Source	16
3.10.3	Constraints	16
3.10.4	Standards	16
3.10.5	Priority	16
3.11	Device Heartbeat and Online Status	16
3.11.1	Description	16
3.11.2	Source	16
3.11.3	Constraints	16
3.11.4	Standards	17
3.11.5	Priority	17
3.12	Secure Multipart Image Ingest	17
3.12.1	Description	17
3.12.2	Source	17
3.12.3	Constraints	17
3.12.4	Standards	17
3.12.5	Priority	17
3.13	Operator Take Picture Command	17
3.13.1	Description	17
3.13.2	Source	17
3.13.3	Constraints	18
3.13.4	Standards	18
3.13.5	Priority	18
3.14	Automatic Plant Upsert and Disease Detection after Pi Capture	18
3.14.1	Description	18
3.14.2	Source	18
3.14.3	Constraints	18
3.14.4	Standards	18
3.14.5	Priority	18

3.15	Edge Device Management UI, Klipper URL, Poses & Schedules	18
3.15.1	Description	18
3.15.2	Source	19
3.15.3	Constraints	19
3.15.4	Standards	19
3.15.5	Priority	19
4	Packaging Requirements	19
4.1	Hardware Packaging	19
4.1.1	Description	19
4.1.2	Source	19
4.1.3	Constraints	19
4.1.4	Standards	19
4.1.5	Priority	20
4.2	Software Packaging	20
4.2.1	Description	20
4.2.2	Source	20
4.2.3	Constraints	20
4.2.4	Standards	20
4.2.5	Priority	20
5	Performance Requirements	20
5.1	Reliability	20
5.1.1	Description	20
5.1.2	Source	20
5.1.3	Constraints	20
5.1.4	Standards	20
5.1.5	Priority	21
5.2	Speed	21
5.2.1	Description	21
5.2.2	Source	21
5.2.3	Constraints	21
5.2.4	Standards	21
5.2.5	Priority	21
6	Safety Requirements	21
6.1	Laboratory Equipment Lockout/Tagout (LOTO) Procedures	21
6.1.1	Description	21
6.1.2	Source	21
6.1.3	Constraints	21
6.1.4	Standards	21
6.1.5	Priority	21
6.2	National Electric Code (NEC) Wiring Compliance	22
6.2.1	Description	22
6.2.2	Source	22
6.2.3	Constraints	22
6.2.4	Standards	22
6.2.5	Priority	22

6.3	RIA Robotic Manipulator Safety Standards	22
6.3.1	Description	22
6.3.2	Source	22
6.3.3	Constraints	22
6.3.4	Standards	22
6.3.5	Priority	22
7	Security Requirements	23
7.1	User Authentication	23
7.1.1	Description	23
7.1.2	Source	23
7.1.3	Constraints	23
7.1.4	Standards	23
7.1.5	Priority	23
7.2	Data Encryption	23
7.2.1	Description	23
7.2.2	Source	23
7.2.3	Constraints	23
7.2.4	Standards	23
7.2.5	Priority	24
7.3	Secure Data Storage	24
7.3.1	Description	24
7.3.2	Source	24
7.3.3	Constraints	24
7.3.4	Standards	24
7.3.5	Priority	24
7.4	Access Control	24
7.4.1	Description	24
7.4.2	Source	24
7.4.3	Constraints	24
7.4.4	Standards	25
7.4.5	Priority	25
7.5	System Integrity and Protection	25
7.5.1	Description	25
7.5.2	Source	25
7.5.3	Constraints	25
7.5.4	Standards	25
7.5.5	Priority	25
8	Maintenance & Support Requirements	25
8.1	System Maintenance Documentation	25
8.1.1	Description	25
8.1.2	Source	25
8.1.3	Constraints	25
8.1.4	Standards	26
8.1.5	Priority	26
8.2	Hardware Maintenance and Replacement	26
8.2.1	Description	26

8.2.2	Source	26
8.2.3	Constraints	26
8.2.4	Standards	26
8.2.5	Priority	26
8.3	Software Updates and Bug Fixes	26
8.3.1	Description	26
8.3.2	Source	26
8.3.3	Constraints	26
8.3.4	Standards	26
8.3.5	Priority	27
8.4	System Monitoring and Error Logging	27
8.4.1	Description	27
8.4.2	Source	27
8.4.3	Constraints	27
8.4.4	Standards	27
8.4.5	Priority	27
8.5	Technical Support Availability	27
8.5.1	Description	27
8.5.2	Source	27
8.5.3	Constraints	27
8.5.4	Standards	27
8.5.5	Priority	27
9	Other Requirements	28
9.1	System Setup and Configuration	28
9.1.1	Description	28
9.1.2	Source	28
9.1.3	Constraints	28
9.1.4	Standards	28
9.1.5	Priority	28
9.2	System Modularity	28
9.2.1	Description	28
9.2.2	Source	28
9.2.3	Constraints	28
9.2.4	Standards	28
9.2.5	Priority	28
9.3	System Extensibility	29
9.3.1	Description	29
9.3.2	Source	29
9.3.3	Constraints	29
9.3.4	Standards	29
9.3.5	Priority	29
9.4	Platform Compatibility	29
9.4.1	Description	29
9.4.2	Source	29
9.4.3	Constraints	29
9.4.4	Standards	29
9.4.5	Priority	29

9.5	Programming Language and Environment	29
9.5.1	Description	29
9.5.2	Source	30
9.5.3	Constraints	30
9.5.4	Standards	30
9.5.5	Priority	30
10	Future Items	30
10.1	Mobile Application Support	30
10.1.1	Description	30
10.1.2	Source	30
10.1.3	Constraints	30
10.1.4	Standards	30
10.1.5	Priority	30
10.2	Advanced AI-Based Plant Disease Detection	30
10.2.1	Description	30
10.2.2	Source	31
10.2.3	Constraints	31
10.2.4	Standards	31
10.2.5	Priority	31
10.3	Cloud-Based Data Analytics and Sharing	31
10.3.1	Description	31
10.3.2	Source	31
10.3.3	Constraints	31
10.3.4	Standards	31
10.3.5	Priority	31
10.4	Offline Device Alerts (AGRI-110)	31
10.4.1	Description	31
10.4.2	Source	31
10.4.3	Constraints	32
10.4.4	Standards	32
10.4.5	Priority	32

List of Figures

1	AgriHome product interfaces: capture hardware, edge agent, API/data plane, and Console.	11
---	---	----

List of Tables

1 Product Concept

1.1 Purpose and Use

AgriHome is a smart plant monitoring product that helps growers and lab operators observe plant health continuously, capture tray imagery from edge hardware, and review diagnoses when computer vision is configured. Users interact primarily through the Vision Console web application. Raspberry Pi devices running Agri-Home/klipper and `fswebcam` (via `agrihome_agent`) act as capture endpoints that auto-provision into trays and deliver images through authenticated device APIs.

1.2 Intended Audience

- Greenhouse operators and facility technicians monitoring multiple trays.
- Home gardeners seeking early visual issue detection without constant manual inspection.
- Educational agricultural programs and senior-design stakeholders evaluating applied CV systems.
- Developers and integrators extending schedules, meshes, or inference backends.

2 Product Description

2.1 Features & Functions

Core product capabilities through Summer 2026 include:

- Authenticated Vision Console (dashboard, trays, plants, mesh, schedule, Devices, settings).
- PostgreSQL-backed domain data for trays, plants, captures, reports, events, and edge devices.
- Automatic and on-demand image capture via edge devices and capture schedules (including destination `raspberrypi-edge`).
- Operator Take Picture with primary Pi agent + `fswebcam` path and optional HTTP streamer still when `klipper_url` is reachable.
- Optional species/disease classification via `cv-backend` / `CV_SPECIES_INFERENCE_URL`, including post-ingest disease detection when `DEVICE_AUTO_DISEASE_ON_INGEST` is enabled (default on).
- Historical monitoring display, alerts/events in-console, and plant detail workflows.
- Secure edge provisioning (`POST /api/raspberrypi/register` + `DEVICE_PROVISIONING_SECRET`; model default `raspberrypi-klipper`), hashed device API keys, heartbeat, and Devices UI (Klipper/streamer URL, poses, schedules).

2.2 External Inputs & Outputs

Inputs: user credentials (Firebase), plant/tray metadata, schedule configuration, agent `fswebcam` JPEGs (and optional HTTP streamer stills), agent heartbeats, multipart image uploads, optional CV inference responses.

Outputs: dashboard health status, capture URLs under `/api/files/...`, plant reports, monitoring events, device online/offline status, queued edge commands, optional classification labels and recommendations.

2.3 Product Interfaces

- Web UI (Next.js App Router PWA) over HTTPS.
- REST API routes under `/api/*` (session cookie for operators; device key header for Pi).
- Optional GraphQL endpoint for selected reads/mutations.
- Agri-Home/klipper on the Pi (`fswebcam` / `save_image.sh`); optional LAN HTTP streamer base stored as `klipper_url`.
- Optional HTTP inference service (`CV_SPECIES_INFERENCE_URL`, tray vision URL).

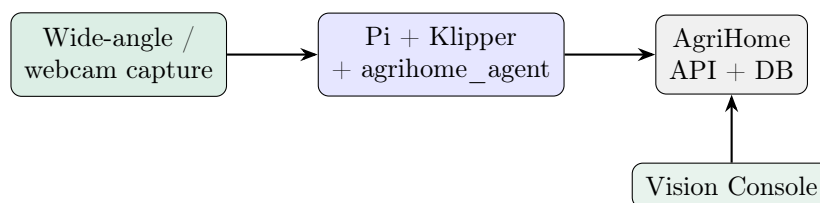


Figure 1: AgriHome product interfaces: capture hardware, edge agent, API/data plane, and Console.

3 Customer Requirements

Customer requirements 3.1–3.9 retain the original SRS intent. Requirements 3.10–3.15 capture the Summer 2026 Raspberry Pi / Klipper edge integration (auto-provisioning, heartbeat, ingest, Take Picture via agent/fswebcam, plant upsert/disease detection, and Devices UI with poses/schedules).

3.1 Plant Health Status Display

3.1.1 Description

The system shall provide a user-facing dashboard that displays the current health status of monitored plants and trays. At minimum, the dashboard shall show a health result such as healthy, warning, or critical, along with the latest available monitoring result for the user to review.

3.1.2 Source

Project Charter, sponsor expectations, and product description.

3.1.3 Constraints

The dashboard must remain simple enough for non-technical users while still showing meaningful monitoring results. Constrained by frontend development time and prototype scope.

3.1.4 Standards

Common dashboard usability and readability practices for software systems.

3.1.5 Priority

Critical

3.2 Automatic Image Capture

3.2.1 Description

The system shall automatically capture plant images at regular intervals for monitoring purposes. Operators shall be able to define capture schedules for trays or meshes. Schedules may target the computer-vision backend path or the `raspberry-pi-edge` destination so that the schedule runner enqueues capture work for provisioned Raspberry Pi devices. This requirement supports reducing constant manual inspection.

3.2.2 Source

Product description, project concept, and Summer 2026 edge integration.

3.2.3 Constraints

Image capture reliability depends on camera/webcam hardware, lighting, plant position, agent `fswebcam` success, optional streamer reachability, agent heartbeat cadence, processing, and storage resources.

3.2.4 Standards

Hardware operating specifications from the selected camera/webcam stack; AgriHome schedule and edge command contracts.

3.2.5 Priority

Critical

3.3 Environmental Data Collection

3.3.1 Description

The system shall collect environmental sensor data associated with plant monitoring when sensors are available. Data may include temperature, humidity, soil moisture, or related conditions used for monitoring and analysis.

3.3.2 Source

Product description and hardware interface requirements.

3.3.3 Constraints

Depends on successful sensor integration, calibration, and communication with the processing unit. Sensor quality and hardware cost may affect completeness and accuracy. Not required for the Pi webcam capture path to function.

3.3.4 Standards

Manufacturer specifications and standard embedded sensor interfacing practices.

3.3.5 Priority

High

3.4 Alert Display

3.4.1 Description

The system shall display alerts or notifications when abnormal plant conditions are detected. Alerts shall be visible through the dashboard/monitoring views and indicate that a plant or tray may require attention.

3.4.2 Source

Product description, customer need, and sponsor expectations.

3.4.3 Constraints

Alert usefulness depends on detection quality and avoiding excessive false positives that cause alarm fatigue.

3.4.4 Standards

Common notification UX practices for monitoring dashboards.

3.4.5 Priority

Critical

3.5 Historical Data Display

3.5.1 Description

The system shall display historical monitoring data so users can review trends over time, including prior captures, reports, and monitoring events associated with trays and plants.

3.5.2 Source

Product description and decision-support goals.

3.5.3 Constraints

Retention is limited by storage capacity and prototype scope; visualization complexity is bounded by UI sprint capacity.

3.5.4 Standards

Standard time-series / history presentation practices for monitoring tools.

3.5.5 Priority

High

3.6 User Settings Input

3.6.1 Description

The system shall allow authenticated users to configure settings relevant to monitoring, such as schedules, preferences (e.g., ML feedback participation), and account session controls.

3.6.2 Source

Product description and usability goals.

3.6.3 Constraints

Settings surface area is constrained to features implemented in the Vision Console; advanced facility-wide RBAC remains future work.

3.6.4 Standards

Standard form validation and authenticated configuration practices.

3.6.5 Priority

High

3.7 Dashboard Interface

3.7.1 Description

The system shall provide a cohesive dashboard interface summarizing tray health, recent imagery when available, and navigation to trays, plants, mesh, schedules, and devices.

3.7.2 Source

Project Charter and customer usability expectations.

3.7.3 Constraints

Must remain responsive on mobile and desktop within the AppShell layout constraints.

3.7.4 Standards

Responsive web / PWA usability guidelines.

3.7.5 Priority

Critical

3.8 No Physical Plant Actuation

3.8.1 Description

The monitoring product shall not perform physical plant interventions such as automated watering, fertilizer dosing, or pesticide application as part of the core prototype scope. Pose/hinge/motor controls on the capture rig, when present, are for camera positioning only and remain stubbed or operator-supervised.

3.8.2 Source

Safety review and project scope boundaries.

3.8.3 Constraints

Hardware motion subsystems must not be presented as autonomous plant-care actuators.

3.8.4 Standards

Laboratory safety practices; RIA-oriented manipulator caution where motion hardware exists.

3.8.5 Priority

Critical

3.9 Remote Notification Support

3.9.1 Description

In a future version, the system should support remote notification delivery such as email, mobile push, or text message when abnormal plant conditions are detected, allowing alerts when the dashboard is not open.

3.9.2 Source

Future customer convenience needs and product expansion goals.

3.9.3 Constraints

Requires additional backend services, notification platform integration, secure internet communication, and account support; may not be feasible within current prototype scope.

3.9.4 Standards

Secure communication practices and standard authentication methods.

3.9.5 Priority

Future

3.10 Edge Device Auto-Provisioning

3.10.1 Description

The system shall auto-provision Raspberry Pi edge devices via `POST /api/raspberry-pi/register`. Agents present a hardware fingerprint (CPU serial required; MAC/hostname/model/`klipperUrl` optional; model defaults to `raspberry-pi-klipper`) and a provisioning code that must match `DEVICE_PROVISIONING_SECRET`. Successful provisioning shall create an `edge_devices` record, issue a device API key (plaintext returned once), store only the SHA-256 hash of that key, and create or link a tray for monitoring. Re-provision of an existing serial shall require an explicit `reProvision` flag.

3.10.2 Source

Summer 2026 Hardware & Software Integration (Pi) sprint; ops threat model AGRI-109.

3.10.3 Constraints

Open registration shall be rejected when `DEVICE_PROVISIONING_SECRET` is unset. Duplicate serial without `reProvision` shall fail safely (HTTP 409). Owner email assignment must follow `DEVICE_DEFAULT_OWNER_EMAIL` or an explicit register payload. Registration must never create a Firebase/human user account.

3.10.4 Standards

Least-privilege device onboarding; secret rotation practices (see Edge Device Security notes).

3.10.5 Priority

Critical

3.11 Device Heartbeat and Online Status

3.11.1 Description

Provisioned devices shall send periodic heartbeats to `POST /api/raspberry-pi/heartbeat` authenticated with the device API key header (`X-Agrihome-Device-Key`). The system shall update last-heartbeat timestamps and derive online/offline (or stale) status for display in the Vision Console. Heartbeat responses shall include pending edge commands for the agent to claim when requested.

3.11.2 Source

Edge reliability and operator situational awareness needs (AGRI-103).

3.11.3 Constraints

Status freshness depends on agent cadence and `DEVICE_HEARTBEAT_STALE_MINUTES` configuration. Network partitions may delay status transitions. Revoked devices must be rejected.

3.11.4 Standards

Standard keep-alive / device presence patterns for IoT-style agents.

3.11.5 Priority

Critical

3.12 Secure Multipart Image Ingest

3.12.1 Description

The system shall accept multipart image uploads from provisioned devices at `POST /api/raspberry-pi/ingest` authenticated with `X-Agrihome-Device-Key`. Uploads shall be size-limited (`DEVICE_INGEST_MAX_BYTES`) and rate-limited per device and per IP (`DEVICE_INGEST_MAX_PER_DEVICE_PER_MIN`, `DEVICE_INGEST_MAX_PER_IP_PER_MIN`). Successful ingest shall persist image bytes under configured storage, create corresponding capture records, and may complete a claimed edge command when a `commandId` is supplied.

3.12.2 Source

Edge capture pipeline; security hardening AGRI-109 / AGRI-101.

3.12.3 Constraints

Device keys are verified against stored SHA-256 hashes; plaintext keys are not re-readable from the database. Revoked devices must be rejected. Storage capacity and max byte limits constrain burst uploads. Excess traffic shall return HTTP 429 with retry guidance.

3.12.4 Standards

TLS in transit where terminated; authenticated upload best practices; rate limiting.

3.12.5 Priority

Critical

3.13 Operator Take Picture Command

3.13.1 Description

Authenticated operators shall be able to trigger Take Picture for a tray-linked edge device from the Vision Console (device action `capture` on `POST /api/devices/{id}`). The primary path shall queue a `capture_now` command for the Pi agent, which captures with `fswebcam` via `Agri-Home/klipper/camera-macros/save_image.sh` and uploads via ingest. When an optional HTTP streamer base is stored as `klipper_url` and reachable from the AgriHome host, the system may attempt a server-side still first (`AGRIHOME_SNAPSHOT_PATH` / timeout configuration) before queuing the agent.

3.13.2 Source

Operator workflow; Raspberry Pi + Klipper integration guide (`docs/ops/RASPBERRY_PI_KLIPPER.md`).

3.13.3 Constraints

Agent capture latency is bounded by heartbeat interval. Optional server-side stills require LAN reachability to the streamer URL. Deprecated aliases `moonrakerUrl` / `updateMoonrakerUrl` remain accepted.

3.13.4 Standards

Agri-Home/klipper camera-macros; AgriHome device action API contract (`updateKlipperUrl`).

3.13.5 Priority

Critical

3.14 Automatic Plant Upsert and Disease Detection after Pi Capture

3.14.1 Description

After a successful Pi capture ingest (or server-side Take Picture persistence), the system shall automatically upsert/attach a plant on the linked tray when plant identity can be determined or created per product rules (pose/slot-aware attach). When `DEVICE_AUTO_DISEASE_ON_INGEST` is enabled (default true), the system shall run leaf species/disease classification asynchronously and persist prediction/report outputs. Optional tray vision may also run when `DEVICE_AUTO_VISION_ON_INGEST` is enabled and inference URLs are configured.

3.14.2 Source

Summer 2026 CV + edge integration (AGRI-107).

3.14.3 Constraints

CV quality depends on model training data and lighting. Auto-disease or auto-vision may be disabled by configuration. Auto-upsert behavior must not corrupt unrelated trays or owners.

3.14.4 Standards

Documented inference JSON contracts for `cv-backend`; data integrity for plant/tray foreign keys.

3.14.5 Priority

High

3.15 Edge Device Management UI, Klipper URL, Poses & Schedules

3.15.1 Description

The Vision Console shall provide edge device management surfaces including a Devices navigation entry listing provisioned devices (serial, status, last heartbeat, linked tray) and a tray-level Raspberry Pi panel supporting Take Picture, device linking, Klipper/streamer URL edit/save (`updateKlipperUrl`; deprecated alias `updateMoonrakerUrl`), and clear empty/offline states. Operators shall be able to configure capture pose sequences for trays and create schedules with destination `raspberry-pi-edge` so the schedule runner enqueues multi-angle `capture_now` work. Device

agents shall obtain active poses and capture plans via authenticated `/api/raspberry-pi/poses` and `/api/raspberry-pi/capture-plan` endpoints.

3.15.2 Source

Operator usability; AGRI-104 / AGRI-105 / AGRI-106.

3.15.3 Constraints

UI must respect authentication and ownership. Klipper URL updates must validate http/https URLs. Offline devices should present clear empty states rather than silent failures. Pose generation depends on tray plant layout data.

3.15.4 Standards

Responsive AppShell navigation patterns already used by the Console.

3.15.5 Priority

High

4 Packaging Requirements

The finished product includes a camera/webcam fixture (optionally mounted with lighting), access to the Vision Console from any modern web browser, and optional AI image processing via a configured inference host. Hardware packaging may require user adjustment of mounting points for shelf attachment. Software packaging is a deployable Next.js application with environment configuration, database migrations, and an edge agent reference package.

4.1 Hardware Packaging

4.1.1 Description

Hardware for the camera system will be partially assembled; mounting the fixture may require adjustment by the user. The package should include custom mounting hardware appropriate to the shelf/tray design. Raspberry Pi + Agri-Home/klipper + `fswebcam` stacks are expected as bench capture units.

4.1.2 Source

Project Charter.

4.1.3 Constraints

Packaging size and assembly complexity restrict how many parts can be pre-assembled.

4.1.4 Standards

None beyond maker-space fabrication norms.

4.1.5 Priority

Medium

4.2 Software Packaging

4.2.1 Description

Access to the main control interface shall be provided as a web application (Vision Console) with documented environment variables, database migrations, and optional container/CI build artifacts. Edge devices shall use the `agrihome_agent` reference package for register/run workflows.

4.2.2 Source

Project Charter and implementation guide.

4.2.3 Constraints

Deployment depends on PostgreSQL availability, Firebase configuration, and network access between Console host and Pi agent; optional when using server-side streamer stills.

4.2.4 Standards

Standard web application packaging and 12-factor style configuration.

4.2.5 Priority

High

5 Performance Requirements

5.1 Reliability

5.1.1 Description

Core monitoring reads/writes shall remain available when optional CV services are down. Edge ingest and heartbeat shall fail closed on invalid credentials. Stale devices shall be markable offline after the configured heartbeat window.

5.1.2 Source

Operational robustness goals.

5.1.3 Constraints

Single-host prototype deployments remain a single point of failure unless operators add redundancy.

5.1.4 Standards

Graceful degradation practices for optional dependencies.

5.1.5 Priority

High

5.2 Speed

5.2.1 Description

Dashboard and tray pages shall load interactive shells within interactive web expectations on lab networks. Take Picture should complete via agent `fswbcam` ingest within a small number of heartbeat intervals under normal bench conditions; when an optional streamer URL is reachable, server-side stills may return an image URL immediately.

5.2.2 Source

Usability goals.

5.2.3 Constraints

WAN latency, WARP/VPN LAN overrides, and Pi CPU load can delay snapshots and uploads.

5.2.4 Standards

Reasonable interactive web performance; no hard real-time guarantees.

5.2.5 Priority

High

6 Safety Requirements

6.1 Laboratory Equipment Lockout/Tagout (LOTO) Procedures

6.1.1 Description

Team members shall follow applicable laboratory LOTO practices when servicing powered fixtures, lighting, or motion hardware associated with the capture rig.

6.1.2 Source

University laboratory safety policy.

6.1.3 Constraints

Student access hours and training availability may limit when hardware can be serviced.

6.1.4 Standards

UTA lab safety / LOTO guidance.

6.1.5 Priority

Critical

6.2 National Electric Code (NEC) Wiring Compliance

6.2.1 Description

Electrical assembly for lighting and device power shall follow safe wiring practices consistent with applicable NEC guidance for the installation context (prototype bench, not a permanent building modification unless approved).

6.2.2 Source

Facilities and safety review.

6.2.3 Constraints

Prototype wiring must remain inspectable and not overload lab circuits.

6.2.4 Standards

NEC-informed safe wiring practices for low-voltage/prototype setups.

6.2.5 Priority

High

6.3 RIA Robotic Manipulator Safety Standards

6.3.1 Description

If hinge/motor positioning hardware is energized, operators shall treat it as motion equipment: maintain clearance, avoid unsupervised autonomous sweeps near people, and keep plant-care actuation out of scope (see requirement 3.8).

6.3.2 Source

Safety scope boundary.

6.3.3 Constraints

Pose control may be stubbed; physical motion capability varies by bench build.

6.3.4 Standards

RIA-oriented manipulator safety caution for educational prototypes.

6.3.5 Priority

High

7 Security Requirements

This section defines security and privacy requirements for AgriHome. The system captures plant images and processes data through backend services. Controls protect user sessions, device credentials, and stored imagery.

7.1 User Authentication

7.1.1 Description

The system shall require user authentication to access the dashboard and protected system data. Users must sign in (Firebase email/password or Google) before accessing plant monitoring information; server routes shall verify session cookies.

7.1.2 Source

System security and user access control requirements.

7.1.3 Constraints

Authentication must remain usable for operators while maintaining adequate security. Password reset and full RBAC remain partially future.

7.1.4 Standards

Standard authentication practices; Firebase Auth guidance.

7.1.5 Priority

Critical

7.2 Data Encryption

7.2.1 Description

Data transmitted between browsers, AgriHome servers, and (where feasible) edge agents shall use TLS when terminated in front of the application. Sensitive secrets must not be committed to source control.

7.2.2 Source

Data protection and privacy requirements.

7.2.3 Constraints

Local bench HTTP may be used on trusted LANs during development; production should prefer TLS.

7.2.4 Standards

TLS for secure data transmission.

7.2.5 Priority

Critical

7.3 Secure Data Storage

7.3.1 Description

Stored data, including plant images and system logs/events, shall be protected against unauthorized access. Access to stored data must be restricted to authorized users. Device API keys shall be stored only as SHA-256 hashes in `edge_devices.api_key_hash`; plaintext keys are returned only once at register or rotate-key and must not be recoverable from the database. Edge device onboarding shall require a rotatable `DEVICE_PROVISIONING_SECRET` (open registration disabled when unset).

7.3.2 Source

Data privacy, system security, and AGRI-109 Edge Device Security notes.

7.3.3 Constraints

Local filesystem storage security depends on host permissions. Database backups must be handled as sensitive. Operators should rotate the provisioning secret when personnel leave or a bench is decommissioned.

7.3.4 Standards

Secure storage best practices; hashed credential storage.

7.3.5 Priority

High

7.4 Access Control

7.4.1 Description

The system shall enforce user-based ownership controls so operators access their trays, plants, schedules, and devices via Firebase session cookies. Device endpoints (`/api/raspberry-pi/*`) shall authenticate with device API keys (`X-Agrihome-Device-Key`) distinct from Firebase user identities. Registration must never create a human account. Operators may revoke or rotate device keys through authenticated device actions.

7.4.2 Source

System design and security requirements; AGRI-109.

7.4.3 Constraints

Fine-grained RBAC roles (admin/reviewer/operator) are not fully implemented in the prototype.

7.4.4 Standards

Access control and authorization best practices.

7.4.5 Priority

High

7.5 System Integrity and Protection

7.5.1 Description

The system shall protect integrity of ingest and command flows via hashed device-key authentication, revoke/rotate key actions, ingest rate limits, maximum upload size, and rejection of invalid provisioning attempts (including replay of registration without intentional re-provision). Unauthenticated HTTP streamers and Klipper APIs should not be exposed to the public Internet without authentication.

7.5.2 Source

Edge threat model: rogue provisioning, stolen keys, spam uploads (AGRI-109).

7.5.3 Constraints

Rate limits and max bytes are configuration-dependent; operators must rotate secrets when benches are decommissioned.

7.5.4 Standards

Defense-in-depth for IoT-style device APIs.

7.5.5 Priority

High

8 Maintenance & Support Requirements

8.1 System Maintenance Documentation

8.1.1 Description

The team shall maintain documentation describing deployment, migrations, edge provisioning, and known networking caveats (e.g., optional streamer URL discovery, WARP LAN overrides, fswebcam device paths).

8.1.2 Source

Course deliverable and operational needs.

8.1.3 Constraints

Docs must track Summer 2026 behavior changes; stale docs are a risk.

8.1.4 Standards

Internal engineering documentation standards.

8.1.5 Priority

High

8.2 Hardware Maintenance and Replacement

8.2.1 Description

Webcam/Pi hardware shall be replaceable by re-provisioning or rotating device credentials. Fixtures should allow camera swap without redesigning the entire rack.

8.2.2 Source

Prototype maintainability.

8.2.3 Constraints

Lead time for replacement parts may delay demos.

8.2.4 Standards

Standard hardware spare strategy.

8.2.5 Priority

Medium

8.3 Software Updates and Bug Fixes

8.3.1 Description

Software shall be updatable via normal git-based development, migrations, and redeploy of the Next.js application and agent package. Critical security fixes for provisioning/ingest shall be prioritized.

8.3.2 Source

Maintenance policy.

8.3.3 Constraints

Course bandwidth limits how many non-critical fixes land each sprint.

8.3.4 Standards

Semantic versioning for documents; conventional deploy practices.

8.3.5 Priority

High

8.4 System Monitoring and Error Logging

8.4.1 Description

The system shall expose health checks (e.g., `/api/health`) reporting database connectivity and whether inference URLs are configured. Edge online status shall be visible in the Devices UI. Application errors should be observable in server logs during operation.

8.4.2 Source

Operability requirements.

8.4.3 Constraints

Centralized log aggregation is optional for the prototype.

8.4.4 Standards

Health-check patterns for web services.

8.4.5 Priority

High

8.5 Technical Support Availability

8.5.1 Description

During the senior-design project window, team members provide support to stakeholders for demos and lab use. Post-course support is not guaranteed unless separately arranged.

8.5.2 Source

Course staffing reality.

8.5.3 Constraints

Availability follows academic calendars and sprint commitments.

8.5.4 Standards

None.

8.5.5 Priority

Medium

9 Other Requirements

9.1 System Setup and Configuration

9.1.1 Description

Operators shall be able to configure environment variables for database, Firebase, storage roots, `DEVICE_PROVISIONING_SECRET`, default owner email, heartbeat stale window, ingest size/rate limits, snapshot path/timeout, `DEVICE_AUTO_DISEASE_ON_INGEST`, `DEVICE_AUTO_VISION_ON_INGEST`, and optional CV URLs before running the application and agent.

9.1.2 Source

Implementation guide and ops docs.

9.1.3 Constraints

Misconfiguration can disable registration or CV/disease paths.

9.1.4 Standards

12-factor configuration practices.

9.1.5 Priority

Critical

9.2 System Modularity

9.2.1 Description

Presentation, application/API, processing (CV), data, and edge agent concerns shall remain separable so that CV or Pi hardware can be absent without blocking core Console CRUD monitoring.

9.2.2 Source

Architectural Design Specification.

9.2.3 Constraints

Some features degrade when optional services are unset.

9.2.4 Standards

Layered architecture modularity.

9.2.5 Priority

High

9.3 System Extensibility

9.3.1 Description

The system shall allow extending schedules, pose sequences, meshes, and inference providers without rewriting the entire Console.

9.3.2 Source

Product roadmap.

9.3.3 Constraints

Extensibility is bounded by schema migrations and API stability.

9.3.4 Standards

Interface-oriented extension practices.

9.3.5 Priority

High

9.4 Platform Compatibility

9.4.1 Description

The Vision Console shall run in modern desktop and mobile browsers. Edge agents target Raspberry Pi OS-class Linux with Python 3 and network access to AgriHome.

9.4.2 Source

Customer accessibility goals.

9.4.3 Constraints

Older browsers and highly locked-down networks may limit PWA install or snapshot reachability.

9.4.4 Standards

Baseline evergreen browser support; Linux edge runtime.

9.4.5 Priority

High

9.5 Programming Language and Environment

9.5.1 Description

Primary application code shall use TypeScript/Next.js with PostgreSQL. CV backend and edge agent shall use Python as implemented. Build/test should run in the team's documented Node and Python environments.

9.5.2 Source

Team skill set and repository standards.

9.5.3 Constraints

Language/runtime upgrades must not be taken lightly mid-semester.

9.5.4 Standards

Language and framework documentation for Next.js, PostgreSQL, and Python.

9.5.5 Priority

High

10 Future Items

This section summarizes features considered but not fully delivered, or only partially implemented, in the current prototype.

10.1 Mobile Application Support

10.1.1 Description

A dedicated native mobile application could allow monitoring, notifications, and settings without a browser. The current PWA covers many mobile needs but is not a store-distributed native app.

10.1.2 Source

User experience enhancement goals.

10.1.3 Constraints

Requires additional development time and cross-platform packaging.

10.1.4 Standards

iOS/Android application guidelines.

10.1.5 Priority

Future

10.2 Advanced AI-Based Plant Disease Detection

10.2.1 Description

Advanced AI-based plant disease detection is **partially implemented**: the team trains and serves a PlantVillage-style multi-class classifier in `cv-backend` and can invoke it from AgriHome when `CV_SPECIES_INFERENCE_URL` is configured, including optional post-Pi-capture detection. Remaining

future work includes broader crop coverage, stronger real-world robustness, richer explainability (e.g., heatmaps), and production MLOps hardening.

10.2.2 Source

Project design discussion; Summer 2026 CV status.

10.2.3 Constraints

Requires datasets, training compute, and careful evaluation under greenhouse lighting.

10.2.4 Standards

AI/ML best practices and data handling guidelines.

10.2.5 Priority

Partial / Future

10.3 Cloud-Based Data Analytics and Sharing

10.3.1 Description

Future versions may add deeper cloud analytics, cross-organization sharing, and multi-site rollups beyond the current single-tenant-style ownership model.

10.3.2 Source

System scalability considerations.

10.3.3 Constraints

Requires additional cloud infrastructure, security review, and ongoing maintenance.

10.3.4 Standards

Cloud computing and data security standards.

10.3.5 Priority

Future

10.4 Offline Device Alerts (AGRI-110)

10.4.1 Description

The system should alert operators when a previously online edge device becomes stale/offline beyond the heartbeat window (in-console first; optionally email/push later). This item is tracked as AGRI-110 and is not fully implemented in Revision 2.1.

10.4.2 Source

Edge operability gap identified during Pi integration.

10.4.3 Constraints

Requires notification policy, deduplication, and ownership-aware delivery.

10.4.4 Standards

Monitoring alert practices; reuse future remote notification channels where available.

10.4.5 Priority

Future

References

- [1] AgriHome, *Project Charter*, CSE 4316 / updated Summer 2026.
- [2] AgriHome, *Architectural Design Specification*, CSE 4316, Spring 2026.
- [3] AgriHome, *Edge Device Security (AGRI-109)*, project operations documentation, 2026.
- [4] AgriHome, *Raspberry Pi + Klipper* operations guide (`docs/ops/RASPBERRY_PI_KLIPPER.md`), 2026.
- [5] AgriHome, *Raspberry Pi + Moonraker* operations guide (superseded stub; see Klipper guide), 2026.
- [6] K. S. Rubin, *Essential Scrum*, Addison-Wesley, 2012.
- [7] Google, “Firebase Authentication,” <https://firebase.google.com/docs/auth>.
- [8] Vercel, Inc., “Next.js Documentation,” <https://nextjs.org/docs>.